

Semestrální práce z KIV/MNS

Organizační aplikace LAN party

Patrik Mrnka
A24B0002P
patmrnka@students.zcu.cz

01.06.2025

Obsah

Obsah	1
1 Úvod	3
1.1 Rozsah projektu	3
1.2 Definice pojmů	3
2 Celkový přehled systému	4
2.1 Architektura systému	4
2.1.1 Koncepční model	4
2.1.2 Hlavní komponenty	4
2.1.3 Komunikační toky	5
2.2 Technologický stack	5
2.2.1 Backend (RespawnApi)	5
2.2.2 Frontend (respawn-app)	6
2.2.3 Ostatní nástroje a technologie	6
2.3 Kontext systému	6
3 Funkční požadavky	8
3.1 Správa uživatelů a autentizace	8
3.2 Správa herních serverů	8
3.3 Správa anket	9
3.4 Správa uživatelských profilů	9
3.5 Sledování přítomnosti uživatelů	9
3.6 Administrátorská sekce	9
4 Nefunkční požadavky	10
4.1 Výkon	10
4.2 Bezpečnost	10
4.3 Spolehlivost	10
5 Návrh systému	11
5.1 Architektura Backendu (RespawnApi)	11
5.1.1 Přehled a adresářová struktura projektu	11
5.1.2 Vrstvy aplikace	12
5.1.3 Použité návrhové vzory	12
5.1.4 Databázový model	15
5.1.5 API Endpoints	17
5.1.6 Autentizace a Autorizace	17
5.1.7 Komunikace v reálném čase (SignalR)	19
5.1.8 Správa Docker kontejnerů	20
5.1.9 Dotazování herních serverů	22
5.1.10 Monitorování stavu serverů	22
5.1.11 Správa Docker Images a Volumes	23
5.2 Architektura Frontend (respawn-app)	24
5.2.1 Přehled a adresářová struktura projektu	24
5.3 Diagramy	24
5.3.1 Diagram tříd - Backend	25
5.3.2 Diagram případů užití	26
5.3.3 Sekvenční diagram: Vytvoření herního serveru	27
5.3.4 Sekvenční diagram: Hlasování v anketě	28

5.3.5	Stavový diagram: Proces vytváření a správy serveru	29
5.3.6	Diagram nasazení	30
6	Implementační detaily	30
6.1	Nastavení vývojového prostředí	30
6.1.1	Požadavky na software	30
6.1.2	Konfigurace API	30
6.1.3	Konfigurace Frontendu	31
6.1.4	Spuštění API a Frontendu pro vývoj	32
6.2	Nasazení	32
7	Uživatelská příručka (stručný přehled)	33
7.1	Registrace a přihlášení	33
7.2	Rozhraní uživatele	33
8	Budoucí vývoj a rozšíření	35
8.1	Připisování skóre a statistiky hráčů	35
8.2	Podpora dalších herních titulů	35
8.3	Zátěžové testování a optimalizace výkonu	35
8.4	Vylepšení UI/UX	35
8.5	Pokročilejší správa Docker prostředků	35
8.6	Možnosti integrace s externími systémy	36
9	Závěr	37
9.1	Shrnutí	37
10	Přílohy	38
10.1	Kompletní specifikace API (Swagger/OpenAPI)	38
10.2	Seznam konfiguračních parametrů	38

1 Úvod

1.1 Rozsah projektu

Aplikace Respawn je navržena pro organizaci offline LAN party. Umožňuje administrátorům snadno vytvářet a spravovat herní servery pro různé hry (aktuálně s primární podporou pro Counter-Strike 1.6) s využitím technologie LinuxGSM a Docker. Uživatelé mohou sledovat stav serverů, jejich statistiky a účastnit se anket.

Klíčové funkcionality zahrnují:

- Správa uživatelů (registrace, přihlášení).
- Vytváření a správa herních serverů (Docker kontejnerů s LinuxGSM).
- Zobrazování statistik herních serverů (stav, hráči, mapa).
- Vytváření anket a hlasování v nich.
- Real-time aktualizace stavů a notifikace pomocí SignalR.

Projekt je stále ve vývoji. Mezi plánované hlavní rozšíření patří implementace systému pro připisování skóre a sledování detailních statistik hráčů na herních serverech.

1.2 Definice pojmů

API Application Programming Interface

ASP.NET Core Open-source framework pro vývoj webových aplikací od Microsoftu

CS 1.6 Counter-Strike 1.6 (počítačová hra)

CRUD Create, Read, Update, Delete (základní databázové operace)

CSS Cascading Style Sheets

DTO Data Transfer Object

EF Core Entity Framework Core (ORM pro .NET)

HTML HyperText Markup Language

HTTP Hypertext Transfer Protocol

JWT JSON Web Token

LAN Local Area Network

LGSM LinuxGSM (nástroj pro správu herních serverů)

ORM Object-Relational Mapper

REST Representational State Transfer

UI User Interface

UX User Experience

VM Virtual Machine

Vue.js Progresivní JavaScriptový framework pro tvorbu UI

SignalR Knihovna pro ASP.NET Core umožňující real-time webovou funkcionality

2 Celkový přehled systému

2.1 Architektura systému

2.1.1 Koncepční model

Aplikace je postavena na vícevrstvé klient-server architektuře.

- **Klient (Frontend):** Jednostránková aplikace (SPA) vytvořená pomocí Vue.js, která komunikuje s backendem přes REST API a WebSockets (SignalR) pro real-time aktualizace. Zodpovídá za uživatelské rozhraní a interakci s uživatelem.
- **Server (Backend):** API postavené na ASP.NET Core, které obsluhuje požadavky z frontendu, spravuje business logiku, komunikuje s databází a Docker enginem pro správu herních serverů.
- **Databáze:** Perzistentní úložiště pro data aplikace (uživatelé, servery, ankety, atd.). Je využívána databáze MySQL.
- **Docker Engine:** Zodpovídá za běh a správu kontejnerizovaných herních serverů (Linux-GSM). Backend komunikuje s Docker API.

Tato architektura umožňuje oddělení zodpovědností, lepší škálovatelnost a udržitelnost.

2.1.2 Hlavní komponenty

1. RespawnApi (Backend):

- **Controllers:** Zpracovávají HTTP požadavky, validují vstup a volají aplikační služby.
- **Services:** Obsahují business logiku, orchestrují operace, komunikují s repository a externími službami (např. Docker).
- **Domain Entities:** Reprezentují klíčové objekty systému (např. `GameServer`, `Poll`, `UserProfile`).
- **DataAccess (Repositories & DbContext):** Abstrahují přístup k databázi pomocí EF Core.
- **Hubs (SignalR):** Zajišťují real-time komunikaci s frontendem.

2. respawn-app (Frontend):

- **Views:** Komponenty reprezentující jednotlivé stránky aplikace.
- **Components:** Znovu použitelné UI prvky.
- **Services:** Zapouzdřují logiku pro komunikaci s backend API a SignalR huby.
- **Stores (Pinia):** Spravují globální stav aplikace (např. autentizační stav).
- **Router (Vue Router):** Zajišťuje navigaci v rámci SPA.

3. Databáze: Ukládá informace o uživateli, herních serverech, anketách, hlasováních, atd.

4. Docker Engine s LinuxGSM images: Prostředí pro běh herních serverů.

2.1.3 Komunikační toky

- **Frontend ↔ Backend (HTTP/REST):** Frontend posílá HTTP požadavky (GET, POST, PUT, DELETE) na backend API pro získání dat, provádění akcí (vytvoření serveru, hlasování v anketě) a autentizaci. Backend odpovídá standardními HTTP status kódy a JSON daty.
- **Frontend ↔ Backend (WebSockets/SignalR):** Pro real-time funkcionality (např. aktualizace stavu serverů, výsledky anket, notifikace o přítomnosti uživatelů, streamování logů serveru) je navázáno trvalé obousměrné spojení pomocí SignalR. Backend může aktivně posílat zprávy připojeným klientům.
- **Backend ↔ Docker Engine:** Backend komunikuje s Docker API (prostřednictvím knihovny `Docker.DotNet`) pro správu životního cyklu Docker kontejnerů (vytváření, spouštění, zastavování, mazání, získávání logů).
- **Backend ↔ Herní Servery (UDP/TCP):** Pro získání detailních informací o běžícím herním serveru (např. seznam hráčů, aktuální mapa pro CS 1.6) backend používá specifické protokoly (např. A2S protokol přes UDP).
- **Backend ↔ Databáze:** Backend komunikuje s databází prostřednictvím EF Core.

2.2 Technologický stack

2.2.1 Backend (RespawnApi)

Backend aplikace je postaven na platformě .NET a ekosystému ASP.NET Core. Klíčové technologie a knihovny jsou:

- **.NET 8 (nynější LTS):** Přesná verze je vždy specifikována v souboru `RespawnApi.csproj`!
- **ASP.NET Core:** Pro tvorbu Web API a real-time funkcionality.
 - **ASP.NET Core MVC / API Endpoints:** Pro definici RESTful API.
 - **ASP.NET Core Identity:** Pro autentizaci a autorizaci uživatelů, správu uživatelských účtů a rolí.
 - **SignalR:** Pro real-time obousměrnou komunikaci mezi serverem a klienty.
- **Entity Framework Core (EF Core):** ORM pro přístup k databázi.
 - `RespawnDbContext.cs` definuje kontext databáze a `DbSet`y pro entity.
 - Definice připojení k databázi viz connection string v `appsettings.json`.
- **JWT (JSON Web Tokens):** Pro stateless autentizaci API. Konfigurace v `Program.cs` pomocí `AddAuthentication(JwtBearerDefaults.AuthenticationScheme).AddJwtBearer(...)`. Služba `TokenService.cs` zodpovídá za generování tokenů.
- **Docker.DotNet:** Knihovna pro programovou interakci s Docker API z .NET kódu.
- **Swagger/OpenAPI (Swashbuckle.AspNetCore):** Pro generování interaktivní dokumentace API. Konfigurace v `Program.cs` pomocí `AddSwaggerGen()` a `UseSwaggerUI()`.

V souborech `Program.cs`, `RespawnApi.csproj`) a konfigurační soubory `appsettings.json` lze najít veškeré detailní informace o použitých balíčcích a jejich konfiguraci!

2.2.2 Frontend (respawn-app)

Frontend je moderní jednostránková aplikace (SPA) postavená na Vue.js.

- **Vue.js 3:** Progresivní JavaScriptový framework.
- **TypeScript:** Staticky typovaný JavaScript pro robustnější kód. Konfigurace v `tsconfig.json`.
- **Vite:** Rychlý buildovací nástroj a vývojový server. Konfigurace v `vite.config.ts`.
- **Pinia:** Oficiální knihovna pro správu stavu ve Vue.js aplikacích (např. `authStore.ts`).
- **Vue Router:** Oficiální knihovna pro routing v rámci SPA. Definuje cesty a navigaci (viz `router/index.txt`).
- **Vuetify 3:** Knihovna UI komponent postavená na Material Design. Inicializace v `main.ts`.
- **Axios:** HTTP klient pro komunikaci s backend API. Používán v servisních vrstvách.
- **@microsoft/signalr:** Klientská knihovna pro SignalR umožňující real-time komunikaci s backendem.

Soubor `package.json` je hlavním zdrojem informací o frontendových závislostech a skriptech.

2.2.3 Ostatní nástroje a technologie

- **Docker:** Platforma pro kontejnerizaci. Využívá se pro izolovaný běh herních serverů. Backend spravuje Docker kontejnery.
- **LinuxGSM (Linux Game Server Managers):** Nástroj pro snadnou instalaci, konfiguraci a správu dedikovaných herních serverů na Linuxu. Respawn využívá Docker obrazy obsahující LinuxGSM pro specifické hry (např. Counter-Strike 1.6).
- **Git:** Distribuovaný systém pro správu verzí, používaný pro vývoj projektu.

2.3 Kontext systému

Aplikace Respawn je navržena tak, aby fungovala jako centrální platforma pro účastníky LAN party. Jejím hlavním účelem je zjednodušit a zefektivnit organizaci herních aktivit.

- **Prostředí nasazení:** Typicky lokální síť (LAN) během konání LAN party. Backend API a frontend jsou dostupné účastníkům připojeným k této síti. Docker Engine běží na serveru v rámci LAN.
- **Uživatelé:**
 - **Administrátoři LAN party:** Mají plnou kontrolu nad vytvářením a správou herních serverů, správou uživatelů a anket.
 - **Správci:** Administrátoři bez pravomocí upravovat uživatele.
 - **Účastníci LAN party (běžní uživatelé):** Mohou si prohlížet dostupné herní servery, jejich stav, připojovat se k nim (mimo aplikaci, pomocí IP a portu), hlasovat v anketách a sledovat svůj profil.
- **Interakce s externími systémy:**
 - **Docker Engine:** Klíčová závislost pro běh herních serverů.
 - **Herní servery (LinuxGSM):** Aplikace bude v příští verzi pracovat s příkazem `LinuxGSM`.

- **Herní klienti (např. Counter-Strike 1.6):** Uživatelé používají své herní klienty pro připojení k serverům vytvořeným aplikací Respawn. Aplikace poskytuje potřebné informace (IP, port).

Cílem je poskytnout hladký a integrovaný zážitek pro organizátory i hráče, minimalizovat manuální konfiguraci serverů a centralizovat informace a interakce spojené s LAN party.

3 Funkční požadavky

Tato sekce popisuje klíčové funkce, které systém Respawn poskytuje uživatelům.

3.1 Správa uživatelů a autentizace

- **1.1 Registrace uživatele:** Systém umožňuje novým uživatelům vytvořit si účet zadáním uživatelského jména, emailu a hesla.
- **1.2 Přihlášení uživatele:** Registrovaní uživatelé se mohou přihlásit do systému pomocí svého uživatelského jména (nebo emailu) a hesla.
- **1.3 Odhlášení uživatele:** Přihlášení uživatelé se mohou odhlásit ze systému.
- **1.4 Správa uživatelských rolí:** Systém podporuje tři role: Administrátor, Správce a Uživatel.
 - Administrátoři mají maximální práva pro správu systému.
 - Přiřazení role Administrátor probíhá manuálně nebo při seedování databáze.
 - Správce má podobná práva, jako Administrátor s rozdílem změn hlavních funkcí aplikace.
- **1.5 Autorizace na základě rolí:** Přístup k určitým funkcím a API endpointům jsou omezeny na základě role uživatele (např. vytváření serverů pouze pro **Administrátory**).

3.2 Správa herních serverů

- **2.1 Vytvoření herního serveru:** Administrátor může vytvořit nový herní server.
 - Musí specifikovat název serveru.
 - Musí vybrat typ hry (např. Counter-Strike 1.6) z dostupných LinuxGSM obrazů.
 - Systém automaticky přiřadí porty a nakonfiguruje Docker kontejner.
 - Stav serveru se aktualizuje (Vytváří se → Online/Chyba).
- **2.2 Zobrazení seznamu herních serverů:** Všichni uživatelé mohou vidět seznam existujících herních serverů, jejich název, typ hry, IP adresu, port a aktuální stav.
- **2.3 Start herního serveru:** Administrátor může spustit zastavený herní server.
- **2.4 Stop herního serveru:** Administrátor může zastavit běžící herní server.
- **2.5 Smazání herního serveru:** Administrátor může smazat existující herní server (včetně asociovaného Docker kontejneru a případných volumes).
- **2.6 Zobrazení detailů herního serveru:** Uživatelé mohou zobrazit detailní informace o serveru, včetně:
 - Aktuální počet hráčů / maximální počet hráčů.
 - Aktuální mapa (pro podporované hry).
 - Seznam připojených hráčů (pro podporované hry).
 - Ping serveru.
 - VŠE ZÁVISÍ NA KONKRÉTNÍM QUERY PROTOKOLU PRO KOMUNIKACI S HERNÍM SERVEREM!
- **2.7 Zobrazení logů herního serveru:** Administrátor může sledovat real-time logy z konzole herního serveru běžícího v Docker kontejneru.

3.3 Správa anket

- **3.1 Vytvoření ankety (Administrátor):** Administrátor může vytvořit novou anketu s otázkou, popisem a definovatelnými možnostmi pro hlasování.
- **3.2 Zobrazení seznamu anket:** Všichni uživatelé mohou vidět seznam aktivních a ukončených anket.
- **3.3 Hlasování v anketě (Uživatel):** Přihlášení uživatelé mohou hlasovat v aktivní anketě výběrem jedné z dostupných možností. Každý uživatel může hlasovat pouze jednou v dané anketě.
- **3.4 Zobrazení výsledků ankety:** Všichni uživatelé mohou vidět aktuální (nebo finální) výsledky hlasování v anketě (počet hlasů pro každou možnost, případně procentuální zastoupení). Výsledky se aktualizují v reálném čase.
- **3.5 Úprava ankety:** Administrátor může upravit název, popis a možnosti ankety, dokud v ní nebylo hlasováno.
- **3.6 Smazání ankety:** Administrátor může smazat anketu.

3.4 Správa uživatelských profilů

- **4.1 Zobrazení uživatelského profilu:** Uživatelé mohou zobrazit základní informace o svém profilu.
- **4.2 Úprava uživatelského profilu:** Uživatelé mohou upravit některé informace ve svém profilu (např. přezdívku).

3.5 Sledování přítomnosti uživatelů

- **5.1 Zobrazení online uživatelů:** Systém zobrazuje seznam aktuálně přihlášených (online) uživatelů v aplikaci.
- **5.2 Real-time aktualizace přítomnosti:** Seznam online uživatelů se aktualizuje v reálném čase, když se uživatelé přihlašují nebo odhlašují (nebo když vyprší jejich session/spojení).
- Zprovozněno pomocí **PresenceHub**.

3.6 Administrátorská sekce

- **6.1 Zobrazení běžících Docker kontejnerů:** Administrátor vidí seznam všech Docker kontejnerů spravovaných aplikací.
- **6.2 Zobrazení dostupných Docker obrazů:** Administrátor vidí seznam relevantních Docker obrazů dostupných lokálně na Docker hostiteli.
- **6.3 Zobrazení Docker volumes:** Administrátor vidí seznam relevantních Docker volumes.

4 Nefunkční požadavky

Tato sekce definuje kvalitativní atributy systému Respawn.

4.1 Výkon

- **1.1 Odezva API:** API endpointy by měly reagovat na běžné požadavky (např. získání seznamu serverů, anket) do 500 ms při typickém zatížení (bude přesněji ověřeno při zátěžovém testu!). Operace náročnější na zdroje (např. vytváření serveru) mohou trvat déle, ale uživatel musí být informován o průběhu.
- **1.2 Real-time aktualizace:** Aktualizace stavů přes SignalR (např. stav serveru, výsledky ankety, seznam online uživatelů) by se měly na frontendu projevit s minimálním zpožděním!
- **1.3 Načítání stránek frontendu:** Klíčové stránky frontendu by se měly načíst a být interaktivní do 3 sekund na běžném hardwaru a síťovém připojení v rámci LAN.

4.2 Bezpečnost

- **2.1 Zabezpečení API:** Všechny citlivé API endpointy musí vyžadovat autentizaci pomocí JWT.
- **2.2 Autorizace:** Přístup k administrátorským funkcím musí být striktně omezen na uživatele s rolí Administrátor.

4.3 Spolehlivost

- **6.1 Stabilita aplikace:** Aplikace (backend i frontend) musí být stabilní a minimalizovat neočekávané pády nebo chyby.
- **6.2 Zpracování chyb:** Systém musí korektně zpracovávat chyby a výjimky, informovat uživatele srozumitelnou formou a logovat detaily pro diagnostiku.
- **6.3 Dostupnost:** Během LAN party by měla být aplikace dostupná s minimálními výpadky.

5 Návrh systému

Tato kapitola detailně popisuje návrh jednotlivých částí systému Respawn.

5.1 Architektura Backendu (RespawnApi)

Backend aplikace **RespawnApi** je navržen s využitím principů vícevrstvé architektury. Cílem je dosáhnout vysoké míry oddělení zodpovědností (Separation of Concerns), testovatelnosti a udržitelnosti.

5.1.1 Přehled a adresářová struktura projektu

Projekt **RespawnApi** je strukturován následovně:

- **/Controllers:** Obsahuje API kontrolery, které slouží jako vstupní body pro HTTP požadavky z frontendu. Zodpovídají za validaci vstupů, volání aplikačních služeb a formátování odpovědí.
- **/Application:** Tato vrstva obsahuje business logiku aplikace.
 - **/DTOs:** Data Transfer Objects používané pro přenos dat mezi vrstvami, zejména mezi kontrolery a aplikačními službami, a také jako kontrakty pro API.
 - **/Services:** Aplikační služby, které orchestrují operace, implementují use-cases a obsahují hlavní business logiku. Mohou využívat doménové služby a repository.
 - **/Interfaces:** Kontrakty pro služby a repository, podporující Dependency Injection.
 - **/Factories:** Momentálně zodpovědné za vybírání strategií pro Query serveru (např. `GameServerInfoStrategyFactory`).
 - **/Strategies:** Implementace různých algoritmů nebo přístupů pro specifické úkoly (např. různé strategie pro získávání informací z herních serverů - `A2SGoldSourceStrategy`).
 - **/Utils:** Pomocné třídy a utility.
- **/Domain:** Jádru aplikace, obsahuje doménové entity, enumerace a případně doménové služby.
 - **/Entities:** Reprezentují klíčové koncepty systému. Obsahují data a chování úzce spojené s danou entitou.
 - **/Enums:** Výčtové typy používané v doméně (např. `GameType`, `ServerStatus`).
- **/DataAccess:** Zodpovídá za perzistenci dat a komunikaci.
 - **/Repositories:** Implementace Repository patternu, poskytující abstrakci nad datovým úložištěm.
 - **RespawnDbContext.cs:** Definuje EF Core DbContext, reprezentující session s databází a mapování entit na databázové tabulky.
- **/Hubs:** Obsahuje SignalR huby pro real-time komunikaci.
- **/Properties:** Obsahuje konfigurační soubory specifické pro spuštění projektu.
- **Program.cs:** Vstupní bod aplikace, konfigurace služeb, middleware pipeline, DI kontejneru.
- **appsettings.json:** Konfigurační soubor aplikace.

5.1.2 Vrstvy aplikace

Architektura se konceptuálně dělí na následující vrstvy:

1. Presentation Layer:

- **Komponenty:** API Kontrolery (`/Controllers`), SignalR Huby (`/Hubs`).
- **Zodpovědnost:** Přijímá požadavky od klienta (HTTP nebo WebSocket), provádí základní validaci, deleguje zpracování na aplikační vrstvu a formátuje odpovědi. Neobsahuje business logiku.

2. Application Layer:

- **Komponenty:** Služby (`/Application/Services`), DTOs (`/Application/DTOs`), Rozhraní pro služby a repository (`/Application/Interfaces`).
- **Zodpovědnost:** Orchestruje business logiku a use-cases. Koordinuje doménové objekty a repository. Definuje kontrakty pro komunikaci s prezentační vrstvou (DTOs). Je nezávislá na UI a na konkrétní implementaci perzistence.

3. Domain Layer:

- **Komponenty:** Doménové entity (`/Domain/Entities`), Výčty (`/Domain/Enums`).
- **Zodpovědnost:** Obsahuje business logiku jádra systému, pravidla a chování specifické pro doménu aplikace. Je zcela nezávislá na ostatních vrstvách.

4. Infrastructure Layer (Infrastrukturní vrstva):

- **Komponenty:** Implementace Repository (`/DataAccess/Repositories`), DbContext (`/DataAccess/RespawnDbContext.cs`), služby pro komunikaci s externími systémy (např. `ContainerManagementService` pro Docker, `TokenService` pro JWT).
- **Zodpovědnost:** Zajišťuje technické detaily jako je přístup k databázi, komunikace se souborovým systémem, sítěmi, nebo jinými externími službami. Implementuje rozhraní definovaná v aplikační nebo doménové vrstvě.

5.1.3 Použité návrhové vzory

V aplikaci je využito několik návrhových vzorů:

• Repository Pattern:

- **Popis:** Odděluje logiku, která načítá data a mapuje je na doménové entity, od business logiky, která s těmito entitami pracuje. Poskytuje abstrakci nad datovým úložištěm.
- **Implementace:** Rozhraní implementující pattern využívající `RespawnDbContext`:
 - * `IGameServerRepository.cs`
 - * `IPollRepository.cs`
 - * `IUserProfileRepository.cs`
- **Výhody:**
 - * Abstrakce datového přístupu, business logika není vázána na konkrétní technologii DB.
 - * Centralizace logiky pro přístup k datům.
- **Nevýhody:**
 - * Může vést k **leaky abstraction**, pokud se snažíme skrýt všechny detaily ORM.

- * Přidává další vrstvu kódu.

- **Unit of Work Pattern:**

- Implicitně s EF Core DbContext!
- **Popis:** Udržuje seznam objektů ovlivněných business transakcí a koordinuje zápis změn a řešení problémů s konkurencí. V EF Core je DbContext sám o sobě implementací Unit of Work.
- **Implementace:** `RespawnDbContext.cs`. Každý HTTP požadavek (typicky) pracuje s jednou instancí DbContext. Metoda `SaveChangesAsync()` na DbContext zapíše všechny sledované změny do databáze v jedné transakci.
- **Výhody:**
 - * Zajišťuje atomicitu operací.
 - * Seskupuje více databázových operací do jedné transakce.
 - * Zlepšuje výkon snížením počtu databázových volání.
- **Nevýhody:**
 - * Životní cyklus DbContext musí být správně spravován.

- **Strategy Pattern:**

- **Popis:** Definuje rodinu algoritmů, zapouzdřuje každý z nich a činí je zaměnitelnými. Umožňuje měnit algoritmus nezávisle na klientech, kteří ho používají.
- **Implementace:** Rozhraní `IGameServerInfoStrategy.cs` definuje kontrakt pro získávání informací z herního serveru. Konkrétní strategie jako:
 - * `A2SGoldSourceStrategy.cs` (CS 1.6)
 - * `NoDetailStrategy.cs` (výchozí stav)
- **Výhody:**
 - * Snadná rozšiřitelnost o nové strategie (podpora nových herních protokolů).
 - * Oddělení logiky pro různé typy her.
 - * Splňuje Open/Closed Principle. (softwarové entity by měly být otevřené pro rozšíření, ale uzavřené pro modifikaci.)
- **Nevýhody:**
 - * Zvyšuje počet tříd/objektů v systému.

- **Factory Pattern:**

- **Popis:** Poskytuje rozhraní pro vytváření objektů v nadřazené třídě, ale umožňuje podtřídám měnit typ vytvářených objektů. V případě aplikace spíše **Simple Factory!**
- **Implementace:** `GameServerInfoStrategyFactory.cs` s metodou `GetStrategy(GameType gameType)`, která na základě typu hry vrací konkrétní instanci `IGameServerInfoStrategy`.
- **Výhody:**
 - * Zapouzdřuje logiku vytváření objektů.
 - * Usnadňuje přidávání nových produktů (strategií) bez modifikace klientského kódu, který továrnu používá.
 - * Klient (např. `GameServerQueryService`) nemusí znát konkrétní implementace strategií, pouze továrnu.

- **Options Pattern:**

- **Popis:** Používá třídy k poskytování silně typovaného přístupu ke skupinám souvisejících nastavení.
- **Implementace:** Konfigurace JWT (např. `JwtSettings`), Docker (`DockerSettings`) a dalších služeb je načítána z `appsettings.json`. Tyto třídy jsou pak registrovány v DI kontejneru pomocí `services.Configure<JwtSettings>` a injektovány pomocí `IOptions<T>`.
- **Výhody:**
 - * Silně typovaný přístup ke konfiguraci.
 - * Oddělení konfigurace od kódu.
 - * Podpora pro sledování změn v konfiguraci (s `IOptionsSnapshot` a `IOptionsMonitor`).
- **Dependency Injection Pattern:**
 - **Popis:** Technika, kde objekt přijímá jiné objekty (závislosti), na kterých závisí, namísto toho, aby si je vytvářel sám. ASP.NET Core má vestavěnou podporu pro DI.
 - **Implementace:** V `Program.cs` jsou služby registrovány v DI kontejneru. Tyto služby jsou pak injektovány do konstruktorů tříd, které je potřebují.
 - **Výhody:**
 - * Zlepšuje udržovatelnost a flexibilitu.
- **Data Transfer Object (DTO) Pattern:**
 - **Popis:** Jednoduché objekty používané k přenosu dat mezi vrstvami, zejména mezi prezentační a aplikační vrstvou, nebo jako kontrakty pro API. Neobsahují žádnou business logiku.
 - **Implementace:** Třídy v adresáři `Application/DTOs`. Jsou používány jako parametry a návratové typy v API kontrolerech a aplikačních službách.
 - **Výhody:**
 - * Snižuje množství dat přenášených mezi vrstvami (přenáší se jen to, co je potřeba).
 - * Odděluje datový kontrakt API od doménových modelů, což umožňuje měnit doménový model bez ovlivnění API (a naopak).
 - **Nevýhody:**
 - * Zvyšuje počet tříd a nutnost mapování mezi DTOs a doménovými entitami. (často řeší AutoMapper!)
- **Background Service:**
 - **Popis:** Pro běh dlouhotrvajících úloh na pozadí, nezávisle na HTTP požadavcích.
 - **Implementace:** `GameServerStatusMonitorService.cs` je registrována jako `IHostedService` v `Program.cs`. Periodicky kontroluje stav herních serverů.
 - **Výhody:**
 - * Umožňuje provádět pravidelné údržbové úlohy nebo monitorování.
 - * Nezatěžuje vlákna zpracovávající HTTP požadavky.
 - **Nevýhody:**
 - * Správa životního cyklu a zpracování chyb v úlohách na pozadí vyžaduje pečlivost.

Návrhy na implementaci dalších vzorů: Pro budoucí rozvoj, zejména s ohledem na plánované statistiky hráčů, by mohly být zváženy následující vzory:

- **CQRS (Command Query Responsibility Segregation):** Oddělení operací pro čtení (Queries) od operací pro zápis (Commands). To může být výhodné pro komplexní dotazy na statistiky a pro optimalizaci výkonu.
- **Mediator Pattern:** Zapouzdřuje, jak sada objektů interaguje. Může zjednodušit komunikaci mezi komponentami v aplikační vrstvě, zejména pokud se počet služeb a jejich závislostí zvýší. Lze použít knihovnu MediatR knihovnu z .NET.
- **Specification Pattern:** Pro zapouzdření komplexních kritérií dotazování doménových entit. Může být použit ve spojení s Repository patternem k vytvoření znovupoužitelných a kombinovatelných dotazovacích logik.

5.1.4 Databázový model

Databázový model aplikace je realizován pomocí Entity Framework Core. Hlavní entity a jejich vztahy jsou definovány v adresáři `Domain/Entities` a konfigurovány v `RespawnDbContext.cs`.

Hlavní entity:

- **IdentityUser (ASP.NET Core Identity):** Standardní entita pro ukládání základních informací o uživateli (Id, UserName, Email, PasswordHash atd.). Není přímo v `Domain/Entities`, ale je základem pro `UserProfile`.
- **UserProfile:** Rozšiřuje `IdentityUser` o aplikačně specifické informace.
 - Klíčové atributy: Id (primární klíč), Nickname, UserId (cizí klíč k `IdentityUser`).
 - Vztahy:
 - * One-to-One s `IdentityUser` (konceptuálně, propojeno přes `UserId`).
 - * One-to-Many s `GameServer` (jako tvůrce serveru, `CreatedBy`).
 - * One-to-Many s `Poll` (jako tvůrce ankety, `CreatedBy`).
 - * One-to-Many s `PollVote` (uživatel, který hlasoval).
 - * One-to-Many s `PlayerStats` (statistiky hráče).
- **GameServer:** Reprezentuje herní server.
 - Klíčové atributy: Id, Name, IPAddress, Port, GameType (enum), Status (enum), ContainerId, MaxPlayers, CurrentPlayers, MapName, CreatedById.
 - Vztahy:
 - * Many-to-One s `UserProfile` (`CreatedBy`).
 - * One-to-Many s `GameSession`.
- **Poll:** Reprezentuje anketu.
 - Klíčové atributy: Id, Title, Description, IsActive, IsMultipleChoice, CreatedById.
 - Vztahy:
 - * Many-to-One s `UserProfile` (`CreatedBy`).
 - * One-to-Many s `PollOption`.
 - * One-to-Many s `PollVote`.
- **PollOption:** Možnost v anketě.

- Klíčové atributy: `Id`, `Text`, `PollId`.
- Vztahy:
 - * Many-to-One s `Poll`.
 - * One-to-Many s `PollVote`.
- **PollVote:** Hlas uživatele v anketě.
 - Klíčové atributy: `Id`, `PollOptionId`, `UserProfileId`, `PollId`. Zajišťuje, že uživatel může hlasovat pro danou možnost v dané anketě pouze jednou (unikátní klíč nad `PollId` a `UserProfileId` je nakonfigurován v `RespawnDbContext`).
 - Vztahy:
 - * Many-to-One s `PollOption`.
 - * Many-to-One s `UserProfile`.
 - * Many-to-One s `Poll`.
- **GameSession:** Reprezentuje jednu herní session na serveru (např. od startu do restartu mapy, nebo delší období).
 - Klíčové atributy: `Id`, `GameServerId`, `StartTime`, `EndTime`.
 - Vztahy:
 - * Many-to-One s `GameServer`.
 - * One-to-Many s `PlayerStats`.
- **PlayerStats:** Statistiky hráče v rámci jedné herní session.
 - Klíčové atributy: `Id`, `UserProfileId`, `GameSessionId`, `Score`, `Kills`, `Deaths`.
 - Vztahy:
 - * Many-to-One s `UserProfile`.
 - * Many-to-One s `GameSession`.

Detailní konfigurace vztahů, indexů a omezení (např. kaskádové mazání, unikátní klíče) je definována v metodě `OnModelCreating` třídy `RespawnDbContext.cs`. Například:

```

1  // Příklad z RespawnDbContext.cs
2  protected override void OnModelCreating(ModelBuilder modelBuilder)
3  {
4      base.OnModelCreating(modelBuilder); // Důležité pro Identity
5
6      // Unikátní hlas pro uživatele v anketě
7      modelBuilder.Entity<PollVote>()
8          .HasIndex(pv => new { pv.PollId, pv.UserProfileId })
9          .IsUnique();
10
11     // Konfigurace kaskádového mazání pro PollOption při smazání Poll
12     modelBuilder.Entity<Poll>()
13         .HasMany(p => p.Options)
14         .WithOne(o => o.Poll)
15         .HasForeignKey(o => o.PollId)
16         .OnDelete(DeleteBehavior.Cascade); // Pokud jsou smazány Poll, smaž i PollOption
17
18     // ... další konfigurace ...
19 }

```

Pro správu změn schématu databáze se využívají EF Core Migrations. Migrace umožňují verzovat změny databázového schématu a aplikovat je na cílovou databázi.

Entity Relationship Diagram (ERD): Konceptuální ERD je částečně znázorněn v UML Class Diagramu (viz obrázek 5.3.1).

5.1.5 API Endpoints

API aplikace je navrženo jako RESTful a poskytuje frontendové aplikaci rozhraní pro veškerou interakci se serverovou logikou. Endpoints jsou definovány v kontrolerech v adresáři `/Controllers` či pomocí Swagger zmíněného v sekci **Přílohy**.

Většina endpointů vyžadujících modifikaci dat nebo přístup k citlivým informacím je chráněna pomocí atributu `[Authorize]`.

5.1.6 Autentizace a Autorizace

Systém využívá ASP.NET Core Identity pro správu uživatelů a rolí a JWT pro autentizaci API požadavků.

ASP.NET Core Identity:

- **Správa uživatelů:** `IdentityUser` je základní třída pro uživatele, rozšířená o `UserProfile` pro aplikačně specifická data.
- **Správa rolí:** `IdentityRole` se používá pro definici rolí.
- **Ukládání hesel:** Identity framework automaticky zajišťuje bezpečné hashování a ukládání hesel.

Konfigurace Identity služeb probíhá v `Program.cs`:

```
1 builder.Services.AddIdentity<IdentityUser, IdentityRole>(options => {
2     options.Password.RequireDigit = true;
3     // ... další nastavení ...
4 })
5 .AddEntityFrameworkStores<RespawnDbContext>()
6 .AddDefaultTokenProviders();
```

JSON Web Tokens (JWT):

- **Generování tokenů:** Po úspěšném přihlášení v `AuthController` je volána služba `ITokenService` k vygenerování JWT. Token obsahuje klíčové informace (claims) o uživateli, jako jsou `UserId`, `UserName`, `Email` a `role`.

```
1 // Příklad z TokenService.cs
2 public async Task<AuthResponseDto> GenerateTokenAsync
3     (IdentityUser user, UserProfile? userProfile)
4     {
5         var jwtSettings = _configuration.GetSection("JwtSettings");
6         // get JWT settings from configuraton
7         var key = Encoding.ASCII.GetBytes(jwtSettings["Key"])
8         // get JWT key from configuration
9         ?? throw new
```

```

10         InvalidOperationException("JWT Key not found in configuration for TokenService.));
11
12     var userRoles = await _userManager.GetRolesAsync(user);
13     // get user roles from UserManager
14
15     var claims = new List<Claim> // create a list of claims for the JWT token
16     {
17         new Claim(JwtRegisteredClaimNames.Sub, user.Id),
18         new Claim(JwtRegisteredClaimNames.Jti, Guid.NewGuid().ToString()),
19         new Claim(JwtRegisteredClaimNames.Email, user.Email ?? string.Empty),
20         new Claim(ClaimTypes.Name, user.UserName ?? string.Empty),
21         new Claim("uid", user.Id)
22     };
23
24     foreach (var userRole in userRoles) // add each user role as a claim
25     {
26         claims.Add(new Claim(ClaimTypes.Role, userRole));
27     }
28
29     var tokenDescriptor = new SecurityTokenDescriptor // create the token descriptor
30     {
31         Subject = new ClaimsIdentity(claims),
32         Expires = DateTime.UtcNow.AddMinutes(Convert.ToDouble(jwtSettings["DurationInMinutes"])),
33         Issuer = jwtSettings["Issuer"],
34         Audience = jwtSettings["Audience"],
35         SigningCredentials = new SigningCredentials(new SymmetricSecurityKey(key),
36             SecurityAlgorithms.HmacSha256Signature)
37     };
38     // ... vytvoření a zápis tokenu ...
39 }

```

Konfigurační parametry pro JWT (Secret Key, Issuer, Audience, Duration) jsou načítány z `appsettings.json` pomocí Options patternu (`JwtSettings`).

- **Validace tokenů:** Middleware ASP.NET Core pro JWT Bearer autentizaci je nakonfigurován v `Program.cs`. Při každém požadavku na chráněný endpoint tento middleware validuje JWT z `Authorization` hlavičky.

```

1 builder.Services.AddAuthentication(options =>
2     {
3         options.DefaultAuthenticateScheme = JwtBearerDefaults.AuthenticationScheme;
4         options.DefaultChallengeScheme = JwtBearerDefaults.AuthenticationScheme;
5         options.DefaultScheme = JwtBearerDefaults.AuthenticationScheme;
6     })
7     .AddJwtBearer(options =>
8     {
9         options.SaveToken = true;
10        options.RequireHttpsMetadata = false; // Set to true in production
11        options.TokenValidationParameters = new TokenValidationParameters
12        {
13            ValidateIssuer = true,
14            ValidateAudience = true,
15            ValidateLifetime = true,

```

```

16         ValidateIssuerSigningKey = true,
17         ValidIssuer = jwtSettings["Issuer"],
18         ValidAudience = jwtSettings["Audience"],
19         IssuerSigningKey = new SymmetricSecurityKey(key),
20         ClockSkew = TimeSpan.Zero // Remove clock skew for precise expiration
21     };
22     options.Events = new JwtBearerEvents
23     {
24         OnMessageReceived = context =>
25         {
26             var accessToken = context.Request.Query["access_token"];
27             var path = context.HttpContext.Request.Path;
28             if (!string.IsNullOrEmpty(accessToken) &&
29                 (path.StartsWithSegments("/pollHub") ||
30                  path.StartsWithSegments("/presenceHub") ||
31                  path.StartsWithSegments("/gameServerHub") ||
32                  path.StartsWithSegments("/serverLogHub")))
33             {
34                 context.Token = accessToken;
35             }
36
37             return Task.CompletedTask;
38         }
39     };
40 });

```

V `OnMessageReceived` je speciální konfigurace pro umožnění autentizace SignalR spojení přes query parametr, jelikož WebSockets standardně nepodporují posílání Authorization hlaviček.

Autorizace:

- **Atribut `[Authorize]`:** Používá se k ochraně API endpointů nebo celých kontrolerů. Vyžaduje, aby byl uživatel autentizován.
- **Role-based autorizace:** Atribut `[Authorize(Roles = "Admin")]` omezuje přístup pouze na uživatele s rolí "Admin". Role jsou definovány v `Domain/Enums/UserRoles.cs` a přiřazovány uživatelům.

5.1.7 Komunikace v reálném čase (SignalR)

SignalR se používá k poskytování real-time aktualizací frontendovým klientům. Aplikace definuje několik hubů, každý pro specifickou oblast funkcionality. Huby fungují jako centrála pro spojení requestů.

Konfigurace SignalR: Služby SignalR jsou registrovány v `Program.cs` a huby jsou mapovány v pipeline pomocí `app.MapHub<THub>("/hubs/hubName")`.

Hlavní Huby:

- **GameServerHub.cs:**
 - **Účel:** Notifikace o změnách stavu herních serverů, seznamu hráčů, aktuální mapě atd.

- Momentálně méně implementovaný!
- **PollHub.cs:**
 - **Účel:** Real-time aktualizace anket a jejich výsledků.
- **PresenceHub.cs:**
 - **Účel:** Sledování online uživatelů v aplikaci.
 - **Metody Hubu:**
 - * **OnConnectedAsync():** Voláno, když se klient připojí. Zaregistruje uživatele v `IUserPresenceService` a notifikuje ostatní klienty.
 - * **OnDisconnectedAsync(Exception exception):** Voláno, když se klient odpojí. Odstraní uživatele z `IUserPresenceService` a notifikuje ostatní.
 - Služba `UserPresenceService.cs` udržuje stav online uživatelů (např. v `ConcurrentDictionary`) a komunikuje s `PresenceHub` přes `IHubContext<PresenceHub>`.
- **ServerLogHub.cs:**
 - **Účel:** Streamování logů z Docker kontejnerů (herních serverů) administrátorům.
 - **Použití:** Služba `ContainerManagementService` při získávání logů z Dockeru může tyto logy posílat do `ServerLogHub` pomocí `IHubContext<ServerLogHub>`, který je pak rozešle relevantním klientům.

Autentizace pro SignalR huby je zajištěna (jak bylo zmíněno v sekci o JWT) pomocí tokenu předaného v query stringu při navazování WebSocket spojení.

5.1.8 Správa Docker kontejnerů

Správa Docker kontejnerů je klíčovou funkcionalitou pro vytváření a provoz herních serverů. Tuto logiku zapouzdřuje primárně `ContainerManagementService.cs`, která využívá knihovnu `Docker.DotNet` pro komunikaci s Docker API.

Proces vytvoření herního serveru (`GameServersController` → `ContainerManagementService`):

1. Administrátor odešle požadavek na vytvoření serveru (pomocí `CreateGameServerDto`, které obsahuje např. název serveru a typ hry, což implikuje Docker image).
2. `GameServersController` validuje požadavek a vytvoří záznam o `GameServer` v databázi se stavem `ServerStatus.Creating`.
3. Volá se `IContainerManagementService.CreateContainerAsync(...)`.
4. **Výběr Docker image:** Na základě typu hry se určí název Docker image (např. `gameservermanagers/gameserver:cs`). Je počítáno s existencí Docker Image!
5. **Konfigurace sítě a portů:** Aplikace konfiguruje Docker kontejnery pro použití síťového režimu "host". To znamená, že kontejner sdílí síťový prostor hostitelského systému a nemá vlastní izolovanou IP adresu; místo toho používá přímo IP adresu hostitele. Herní server běžící uvnitř kontejneru tak přímo naslouchá na portu na IP adrese hostitelského stroje. Aplikace při vytváření serveru (např. pro Counter-Strike 1.6) nastaví entitě `GameServer` výchozí port pro danou hru (např. 27015/udp). Tento port musí být volný na hostitelském stroji. Díky režimu "host" není prováděno explicitní mapování portů. Konfigurace portu může být případně dále upřesněna pomocí environmentální proměnné `GS_PARAMS` předávané kontejneru.

```

1      var createParams = new CreateContainerParameters // creating container parameters
2  {
3      Image = imageName,
4      Name = containerName,
5      Env = envVars,
6      Labels = new Dictionary<string, string>
7      {
8          { "com.respawn.gameserver.id", serverDetails.GameServerId.ToString() },
9          { "com.respawn.lgsm.volume", volumeName }
10     },
11     HostConfig = new HostConfig
12     {
13         RestartPolicy = new RestartPolicy { Name = RestartPolicyKind.UnlessStopped },
14         NetworkMode = "host",
15         Binds = new List<string> { $"{volumeName}:/data" }
16     },
17 };

```

Informace o IP adrese (nastavená na adresu Docker hosta z konfigurace) a portu (např. výchozí port hry) je následně uložena k entitě `GameServer` v databázi.

6. **Environmentální proměnné:** Pro konfiguraci LinuxGSM serveru uvnitř kontejneru se mohou předávat environmentální proměnné (např. název serveru, RCON heslo, parametry spuštění).
7. **Vytvoření kontejneru:** Pomocí `dockerClient.Containers.CreateContainerAsync(...)` se vytvoří kontejner s danou konfigurací.
8. `ContainerId` je uložen k entitě `GameServer` v databázi.
9. **Spuštění kontejneru:** `dockerClient.Containers.StartContainerAsync(...)`.
10. Stav serveru v DB se aktualizuje (např. na `ServerStatus.Starting`). Monitorovací služba později potvrdí `Online`.

Správa životního cyklu:

- **Start/Stop:** `GameServersController` volá metody `IContainerManagementService` jako `StartContainerAsync`, `StopContainerAsync`. Restart může být implementován jako stop následovaný startem.u
- **Smazání:** Při smazání `GameServer` se volá `RemoveContainerAsync(containerId, new ContainerRemoveParameters Force = true, RemoveVolumes = true )`. Parametr `RemoveVolumes = true` zajistí smazání anonymních volumes; pojmenované volumes je třeba spravovat explicitněji (např. přes `IVolumeManagementService`).

Získávání logů:

- `ContainerManagementService.GetContainerLogsAsync(containerId, ...)` používá `dockerClient.Containers.GetContainerLogsAsync(...)`.
- Logy jsou streamovány. Služba může tyto logy číst a posílat je přes `ServerLogHub` připojeným klientům (administrátorům).

5.1.9 Dotazování herních serverů

Pro získání detailních informací o běžícím herním serveru (počet hráčů, mapa, seznam hráčů) se využívá služba `GameServerQueryService.cs`. Tato služba implementuje Strategy pattern pro podporu různých herních protokolů.

Komponenty:

- **IGameServerQueryService:** Rozhraní definující metodu `GetServerDetailsAsync(ipAddress, port, gameType)`.
- **GameServerQueryService:** Implementace `IGameServerQueryService`. Využívá `IGameServerInfoStrategyFactory` k získání správné strategie pro daný `GameType`.
- **IGameServerInfoStrategyFactory:** Továrna (`GameServerInfoStrategyFactory.cs`) vracující konkrétní implementaci strategie.
- **IGameServerInfoStrategy:** Rozhraní pro strategie, definuje metodu `GetServerInfoAsync(ipAddress, port)`.
- **A2SGoldSourceStrategy.cs:** Strategie pro hry běžící na GoldSource engine (např. Counter-Strike 1.6), které podporují A2S (Server Query Protocol od Valve).
 - Komunikuje přes UDP.
 - Odesílá specifické bajtové sekvence pro různé typy dotazů:
 - * **A2S_INFO:** Základní informace (název serveru, mapa, počet hráčů, max hráčů, OS, typ serveru).
 - * **A2S_PLAYER:** Seznam hráčů (jméno, skóre, doba připojení).
 - * **A2S_RULES:** Pravidla serveru (proměnné serveru). (Není používáno, ale je součástí protokolu).
 - Odpovědi jsou binární data, která je třeba parsrovat. K tomu slouží pomocná třída `BinaryDataParser.cs`, která obsahuje metody pro čtení různých datových typů (byte, short, int, float, string) z bajtového pole.
 - Příklad A2S_INFO dotazu (bajty se mohou lišit): 0xFF, 0xFF, 0xFF, 0xFF, 0x54, 0x53, 0x6F, 0x75, 0x72, 0x63, 0x65, 0x20, 0x45, 0x6E, 0x67, 0x69, 0x6E, 0x65, 0x20, 0x51, 0x75, 0x65, 0x72, 0x79, 0x00.
 - Posílané bajty lze sledovat pomocí programu **WireShark!**
- **NoDetailStrategy.cs:** Výchozí strategie pro hry, kde detailní dotazování není implementováno nebo podporováno. Vrací základní informace dostupné z `GameServer` entity bez externího dotazu.

Výsledkem dotazu je typicky `GameServerDetailDto`, který obsahuje agregované informace.

5.1.10 Monitorování stavu serverů

Periodické monitorování stavu herních serverů zajišťuje `GameServerStatusMonitorService.cs`, která je implementována jako `IHostedService` (Background Service).

Princip fungování:

1. Služba se spouští na pozadí při startu aplikace.
2. V pravidelných intervalech (momentálně 15 sekund) provádí následující kroky...
3. Získá seznam všech aktivních (nebo relevantních) herních serverů z databáze pomocí `IGameServerRepository`.
4. Pro každý server:
 - **Ověření stavu kontejneru:** Kontaktuje Docker API (přes `IContainerManagementService` nebo přímo) a zjistí aktuální stav Docker kontejneru asociovaného se serverem (např. běžící, zastavený, neexistující).
 - **Dotaz na herní server:** Pokud kontejner běží a server by měl být online, použije `IGameServerQueryService` k získání aktuálních detailů (počet hráčů, mapa atd.).
 - **Aktualizace databáze:** Porovná získané informace s informacemi v databázi. Pokud dojde ke změně (např. stav serveru, počet hráčů), aktualizuje záznam `GameServer` v DB.
 - **Notifikace klientů:** Pokud došlo ke změně stavu nebo klíčových parametrů, odešle notifikaci přes `GameServerHub` (pomocí `IHubContext<GameServerHub>`) všem připojeným klientům. To zajistí, že UI na frontendu zobrazuje aktuální informace.
5. Zpracování chyb: Služba by měla robustně zpracovávat chyby (např. nedostupnost Docker API, timeout při dotazu na herní server) a logovat je, aniž by došlo k jejímu selhání.

Tento mechanismus zajišťuje, že stav serverů v aplikaci je relativně aktuální i bez explicitní akce uživatele.

5.1.11 Správa Docker Images a Volumes

Kromě správy kontejnerů poskytuje aplikace administrátorům také přehled o Docker obrazech a volumes prostřednictvím `DockerAdminController`.

- **`IImageManagementService.cs` / `ImageManagementService.cs`:**
 - Zodpovídá za operace související s Docker obrazy.
 - Metoda volá `dockerClient.Images.ListImagesAsync(...)` k získání seznamu lokálně dostupných obrazů.
 - Mapuje výsledky na `DockerImageDto`.
 - V budoucnu by mohla zahrnovat funkce jako stahování obrazů (`CreateImageAsync`), mazání obrazů (`DeleteImageAsync`) nebo prohledávání Docker Hubu.
- **`IVolumeManagementService.cs` / `VolumeManagementService.cs`:**
 - Zodpovídá za operace související s Docker volumes.
 - Metoda volá `dockerClient.Volumes.ListAsync()` k získání seznamu volumes.
 - Mapuje výsledky na `DockerVolumeDto`.
 - V budoucnu by mohla zahrnovat funkce jako vytváření (`CreateVolumeAsync`), mazání (`RemoveVolumeAsync`) nebo inspekci volumes.

Tyto služby jsou využívány `DockerAdminController` k poskytnutí informací administrátorům.

5.2 Architektura Frontend (respawn-app)

Frontend aplikace je moderní jednostránková aplikace (SPA) postavená na Vue.js 3 s využitím TypeScriptu.

5.2.1 Přehled a adresářová struktura projektu

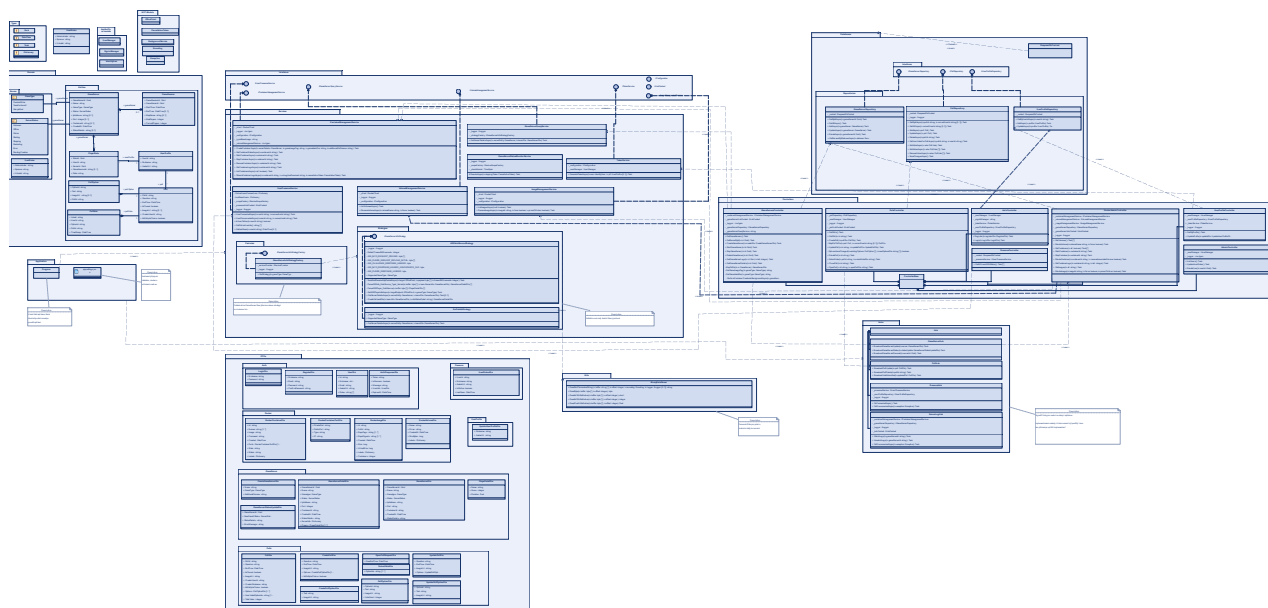
- `/public`: Statické assety, které se přímo kopírují do kořene výstupního adresáře.
- `/src`: Hlavní adresář se zdrojovým kódem aplikace.
 - `/assets`: Statické zdroje, které jsou zpracovávány buildovacím procesem (CSS, obrázky, fonty). Např. `futuristic.css`.
 - `/components`: Znovupoužitelné Vue komponenty, které nejsou specifické pro konkrétní stránku.
 - `/composables`: Funkce Composition API pro sdílení reaktivní logiky mezi komponentami.
 - `/router`: Konfigurace Vue Routeru.
 - * `index.ts`: Definuje cesty, mapování komponent na cesty, navigation guards.
 - `/services`: Služby pro komunikaci s backend API a zapouzdření jiné externí logiky.
 - * `authService.ts`: Pro autentizační požadavky.
 - * `gameServerService.ts`: Pro operace s herními servery.
 - * `pollService.ts`: Pro operace s anketami.
 - * `dockerAdminService.ts`: Pro administrátorské operace s Dockerem.
 - * `profileService.ts`: Pro správu uživatelských profilů.
 - * `adminService.ts`: Pro obecné administrátorské funkce.
 - * `signalrService.ts`: Základní služba pro SignalR spojení.
 - * `serverLogSignalrService.ts`: Specifická služba pro ServerLogHub.
 - * `presenceSignalrService.ts`: Specifická služba pro PresenceHub.
 - `/stores`: Pinia stores pro globální správu stavu.
 - * `authStore.ts`: Spravuje stav autentizace, informace o uživateli, JWT token.
 - `/types`: Typové definice a rozhraní.
 - * `enum.ts`: Výčtové typy (např. pro stavy serverů, typy her, pokud jsou sdíleny s frontendem).
 - * `dockerAdmin.ts`: Typy pro Docker administraci.
 - * Další soubory pro DTOs z backendu...
 - `/views`: Vue komponenty, které reprezentují celé stránky nebo hlavní pohledy aplikace, typicky mapované na cesty v routeru.
 - `App.vue`: Kořenová komponenta aplikace.
 - `main.ts`: Vstupní bod frontendové aplikace, inicializuje Vue, Pinia, Router, Vuetify a další pluginy.

5.3 Diagramy

Tato sekce obsahuje diagramy znázorňující různé aspekty systému, včetně interakcí a stavů.

5.3.1 Diagram tříd - Backend

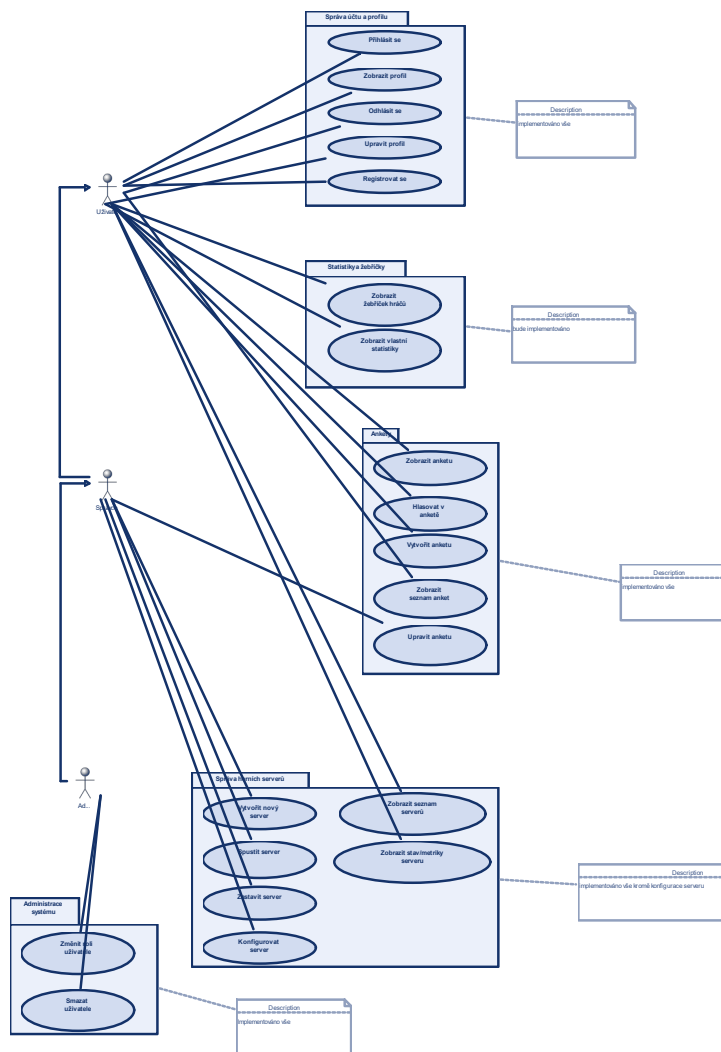
Následující diagram tříd (5.3.1) znázorňuje klíčové entity, služby, repository a kontrolery v backendové aplikaci **RespawnApi** a jejich vzájemné vztahy.



Obrázek 1: Diagram tříd

5.3.2 Diagram případů užití

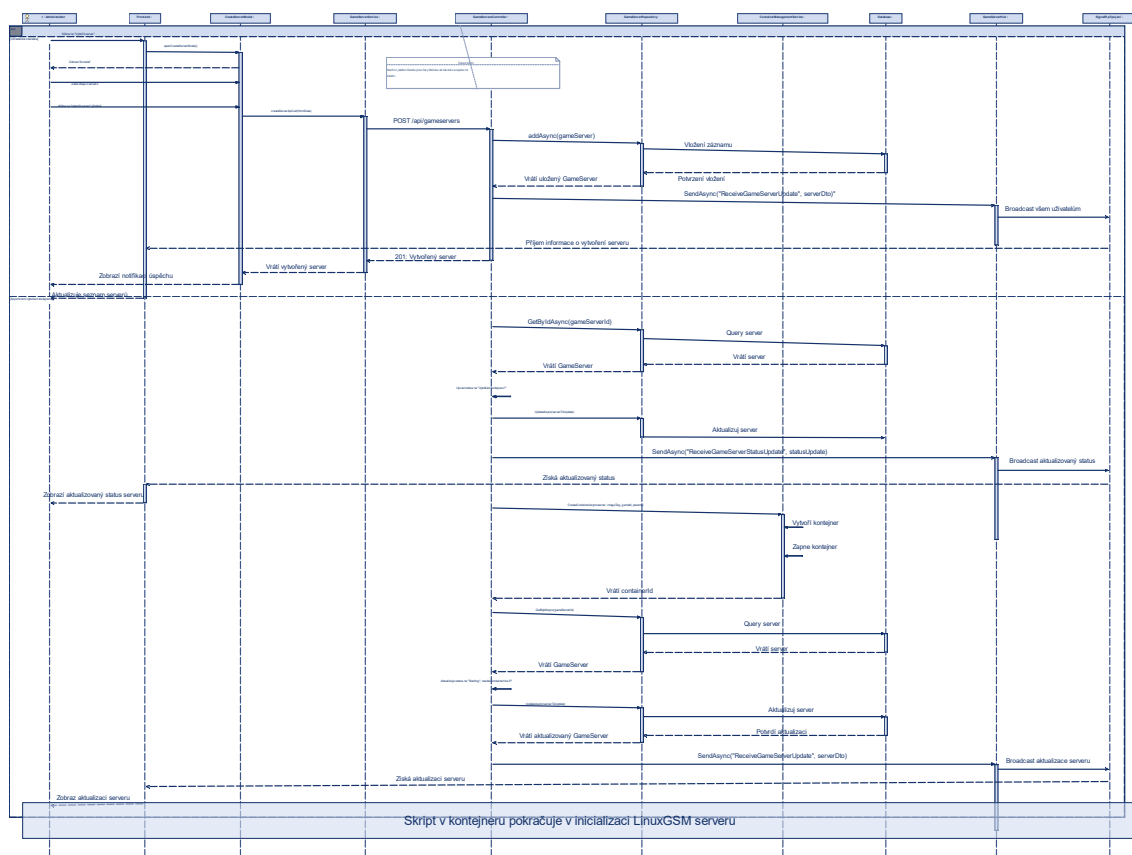
Diagram (obrázek 5.3.2) znázorňuje práva jednotlivých aktérů v aplikaci (administrátor, správce, uživatel), jaké procesy mohou vykonávat.



Obrázek 2: Diagram případů užití

5.3.3 Sekvenční diagram: Vytvoření herního serveru

Tento diagram (obrázek 5.3.3) popisuje sekvenci kroků a interakcí mezi komponentami systému při vytváření nového herního serveru administrátorem.

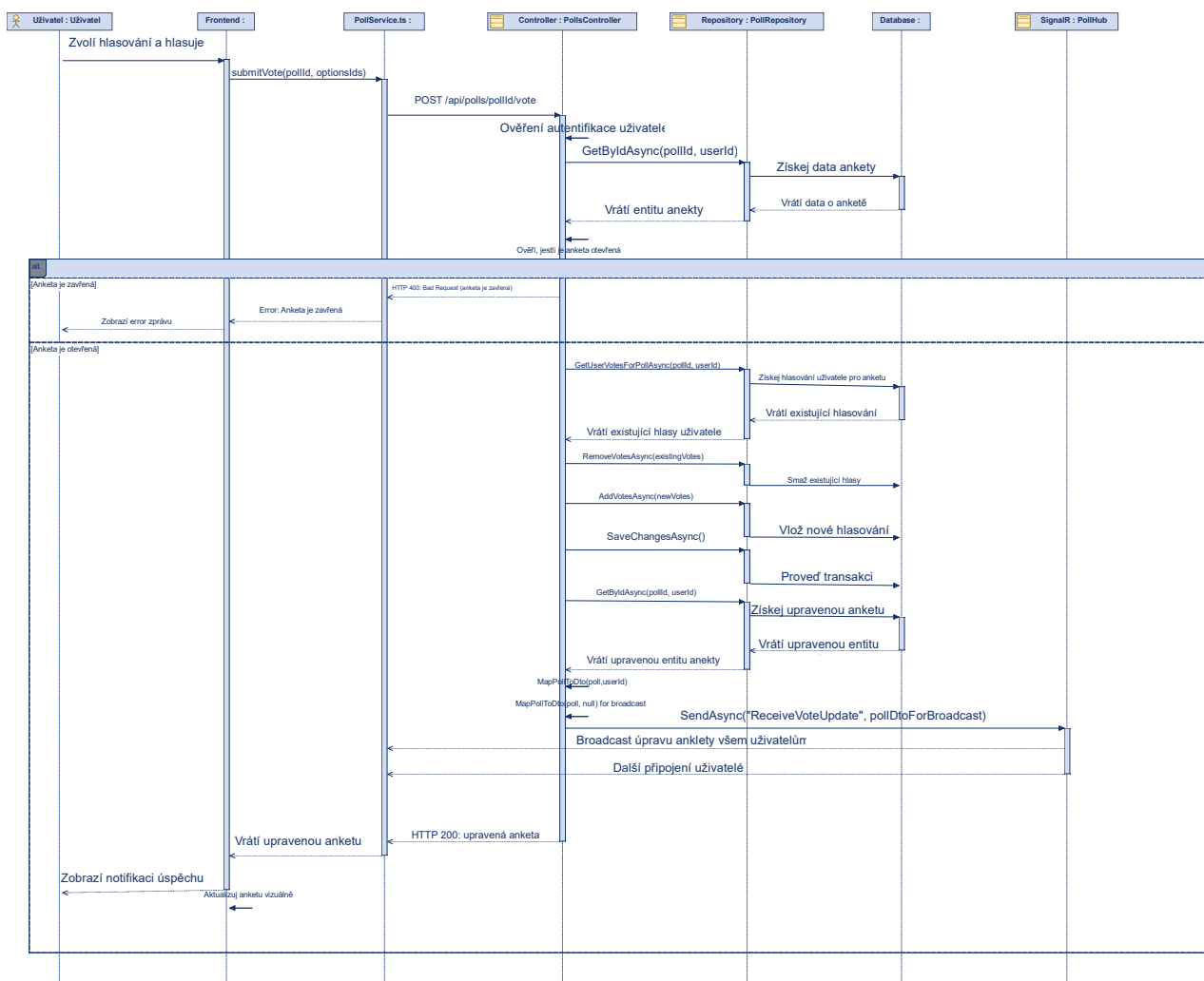


Obrázek 3: Sekvenční diagram - Vytvoření herního serveru

Diagram ukazuje tok od uživatelské akce ve frontendu, přes volání backend API, interakci s databází a Docker engine, až po spuštění LinuxGSM skriptu v kontejneru a zpětnou notifikaci frontendu (včetně SignalR).

5.3.4 Sekvenční diagram: Hlasování v anketě

Následující diagram (obrázek 5.3.4) ilustruje proces hlasování uživatele v anketě, včetně validace a real-time aktualizace výsledků.

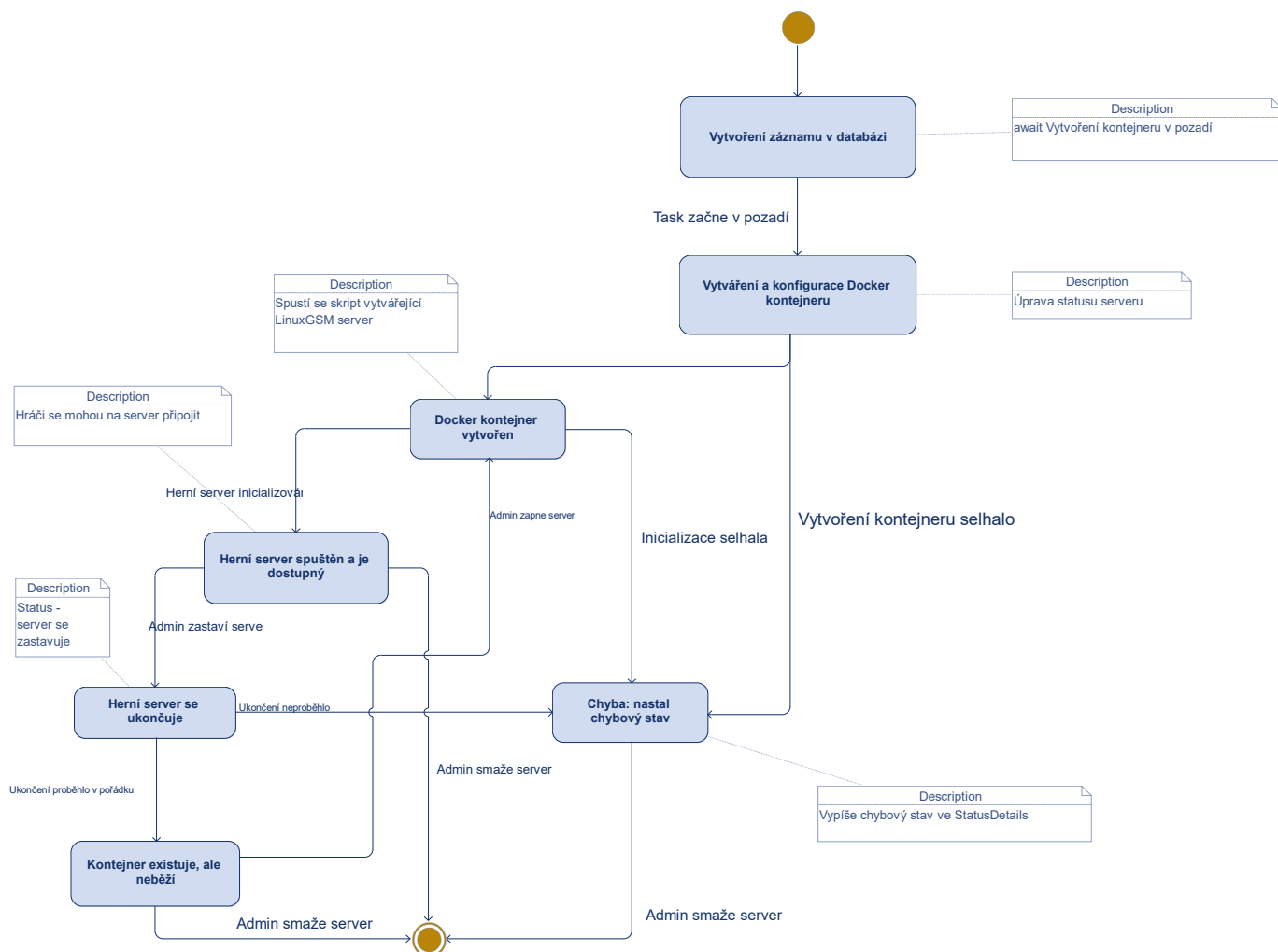


Obrázek 4: Sekvenční diagram - Hlasování v anketě

Tento diagram znázorňuje interakci uživatele s frontendem, volání API pro odeslání hlasu, validaci a uložení hlasu v backendu a následnou aktualizaci UI, včetně real-time notifikace přes SignalR.

5.3.5 Stavový diagram: Proces vytváření a správy serveru

Stavový diagram (obrázek 5.3.5) ukazuje různé stavy, kterými může herní server procházet od svého vytvoření až po smazání, včetně možných přechodů a chybových stavů.

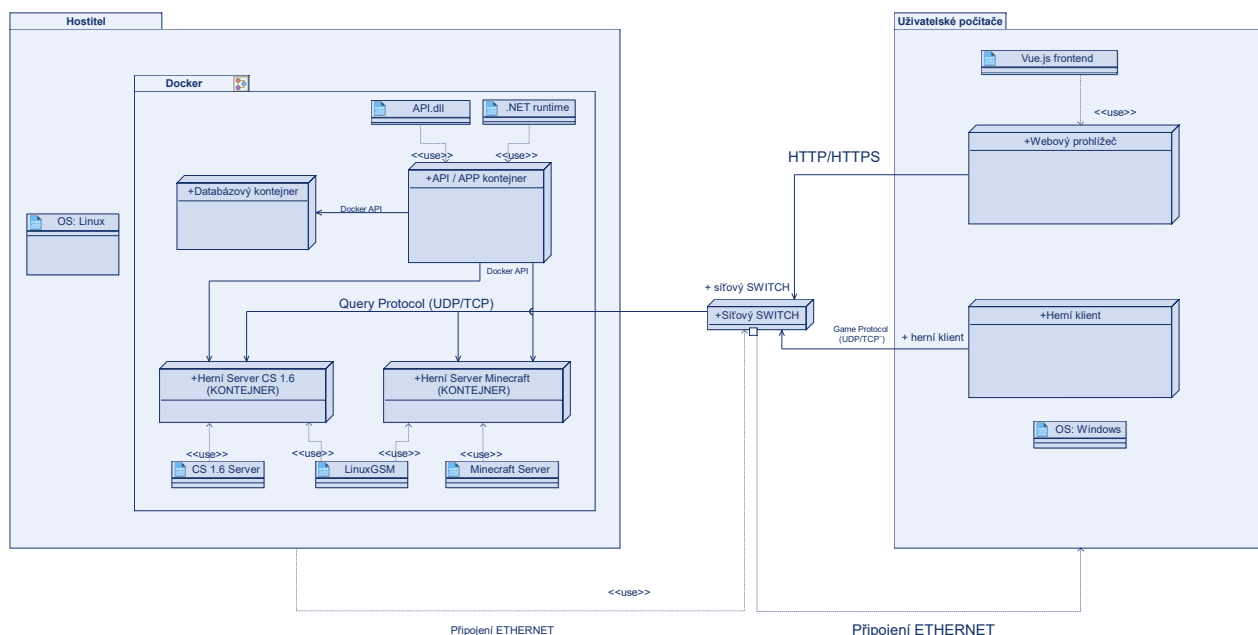


Obrázek 5: Stavový diagram - Životní cyklus herního serveru

Diagram ilustruje komplexní životní cyklus herního serveru, zahrnující stavy jako **Creating**, **Online**, **Offline**, **Error**, a přechody mezi nimi iniciované uživatelskými akcemi nebo systémovými procesy.

5.3.6 Diagram nasazení

Diagram (obrázek 5.3.6) ukazuje výchozí nasazení aplikace do produkce se souvisejícím SW a HW.



Obrázek 6: Diagram nasazení

6 Implementační detaily

Tato kapitola se zabývá praktickými aspekty nastavení vývojového prostředí, sestavení a nasazení aplikace.

6.1 Nastavení vývojového prostředí

6.1.1 Požadavky na software

Pro vývoj a spuštění aplikace jsou vyžadovány následující nástroje:

- **.NET 8 SDK**
- **Node.js:** Pro správu závislostí a spouštění skriptů frontendové aplikace.
- **Docker Engine:** Pro běh a správu Docker kontejnerů.
- **Databázový server** MySQL.

6.1.2 Konfigurace API

1. **Konfigurační soubory:** Hlavní konfigurace se nachází v `RespawnApi/appsettings.json`.
2. **Connection String:**
 - Upravit dle nynější instance:

```

1  "ConnectionStrings": {
2      "DefaultConnection":
3      "server=localhost;
4      port=3306;
5      database=lanpartydb;
6      user=lanparty_user;
7      password=uzivatelske_heslo"
8  },
9  "MySQLSettings": {
10     "ServerVersion": "8.0.42"
11 }

```

3. JWT Secret:

```

1  "JwtSettings": {
2      "Key": "TotoJeVelmiBezpecnyKlickteryMaRozhodneViceNez32BytuProHmacSha256",
3      "Issuer": "RespawnApi",
4      "Audience": "RespawnApp",
5      "DurationInMinutes": 60
6  }

```

Důležité: Pro produkční nasazení `JwtSettings:Key` nikdy neukládat přímo do `appsettings.json`, místo toho se použijí environmentální proměnné!

- Docker Endpoint:** Knihovna `Docker.DotNet` se typicky připojuje k Docker enginu na základě standardních environmentálních proměnných (`DOCKER_HOST`) nebo výchozích socketů/named pipes. Pokud je Docker engine na jiném stroji nebo vyžaduje specifickou konfiguraci, může být potřeba upravit způsob inicializace `DockerClient` v `ContainerManagementService.cs` nebo konfigurovat Docker hosta.

- Aplikování migrací:** Otevřete terminál v adresáři `RespawnApi/RespawnApi`:

```

1  dotnet ef database update

```

Toto vytvoří databázové schéma na základě EF Core migrací.

6.1.3 Konfigurace Frontendu

- Instalace závislostí:** Otevřete terminál v adresáři `respawn-app`:

```

1  npm install

```

- Environmentální proměnné:** Vytvořte soubor `.env` v kořenovém adresáři `respawn-app` pro přepsání výchozích hodnot.

```

1  # Příklad obsahu .env
2  VITE_API_BASE_URL=http://localhost:5207 # Nebo adresa, kde běží vaše API

```

Pokud `VITE_API_URL` není specifikováno, aplikace může používat relativní cestu `/api` (pokud je frontend servírován ze stejné domény jako backend, např. pomocí reverzní proxy v produkci). Výchozí URL API je typicky definováno v `.env` nebo `.env.development`.

6.1.4 Spuštění API a Frontendu pro vývoj

1. Spuštění Backendu (RespawnApi):

- Z IDE: Spustíte projekt.
- Z příkazové řádky (v adresáři `RespawnApi/RespawnApi`):

```
1 dotnet watch run
```

API by mělo běžet na adrese specifikované v `Properties/launchSettings.json` (např. `https://localhost:5207`).

2. Spuštění Frontendu (respawn-app):

- Z příkazové řádky (v adresáři `respawn-app`):

```
1 npm run dev
```

Frontendová aplikace by měla běžet na adrese uvedené ve výstupu Vite (např. `http://localhost:5173`).

Po spuštění obou částí by měla být aplikace dostupná v prohlížeči na adrese frontendového vývojového serveru.

6.2 Nasazení

Pro účely nasazení bude vytvořen `docker-compose.yml` soubor, který bude obsahovat definované **services** s API, APP a databází.

7 Uživatelská příručka (stručný přehled)

Tato sekce poskytuje základní návod pro uživatele a administrátory aplikace Respawn.

7.1 Registrace a přihlášení

1. Registrace:

- Přejděte na registrační stránku přes tlačítko "Registrovat se".
- Vyplňte požadované údaje: uživatelské jméno, emailovou adresu a heslo. Potvrďte heslo.
- Klikněte na tlačítko "Registrovat".
- Po úspěšné registraci můžete být automaticky přihlášení nebo přesměrováni na přihlašovací stránku.

2. Přihlášení:

- Přejděte na přihlašovací stránku.
- Zadejte své uživatelské jméno (nebo email) a heslo.
- Klikněte na tlačítko "Přihlásit se".
- Po úspěšném přihlášení budete přesměrováni na hlavní stránku aplikace.

3. Odhlášení:

- V navigačním menu nebo v uživatelském profilu najdete tlačítko/odkaz "Odhlásit se".
- Kliknutím se odhlásíte a budete přesměrováni na přihlašovací stránku nebo hlavní stránku pro nepřihlášené uživatele.

7.2 Rozhraní uživatele

Po přihlášení má běžný uživatel přístup k následujícím funkcím:

- **Domovská stránka (/home):** Přehled aktuálních informací, uvítací zpráva.
- **Přehled serverů (/servers):**
 - Zobrazení seznamu všech dostupných herních serverů.
 - Pro každý server jsou viditelné informace jako název, typ hry, IP adresa, port, aktuální stav (Online, Offline, Počet hráčů/Max hráčů, Mapa).
 - Možnost kliknout na server pro zobrazení detailů (/servers/:id).
- **Detail serveru (/servers/:id):**
 - Podrobnější informace o serveru, včetně seznamu aktuálně připojených hráčů (pokud je podporováno).
 - Informace pro připojení ke hře (IP:Port).
- **Ankety (/polls):**
 - Zobrazení seznamu aktivních anket.
 - Možnost hlasovat v aktivní anketě výběrem jedné z možností.
 - Zobrazení výsledků již ukončených anket nebo průběžných výsledků aktivních anket.
- **Uživatelský profil (/profile):**

- Zobrazení vlastních profilových informací.
 - Možnost upravit některé údaje (např. přezdívkou).
- **Seznam online uživatelů:** V pravé postranním panelu je zobrazen seznam aktuálně přihlášených uživatelů v aplikaci.

8 Budoucí vývoj a rozšíření

8.1 Připisování skóre a statistiky hráčů

Toto je plánovaný hlavní cíl dalšího vývoje.

- **Sběr dat ze serverů:**
 - **RCON (Remote Console):** Pro hry jako Counter-Strike 1.6 lze použít RCON k posílání příkazů na server (např. `status`, `stats`) a parsování výstupu. Vyžaduje konfiguraci RCON hesla na herním serveru.
 - **Log Parsing:** Mnoho herních serverů generuje detailní logy událostí (zabití, připojení/odpojení hráčů, změny mapy). Backend by mohl tyto logy (z Docker kontejnerů) periodicky číst a parsovat pro extrakci relevantních statistik. To může být náročnější na implementaci kvůli různým formátům logů.
 - **Specializované pluginy/mody pro servery:** Pro některé hry mohou existovat serverové pluginy (např. AMX Mod X pro GoldSource), které mohou exportovat statistiky přes API.
- **Ukládání a zpracování statistik:**
 - Agregace statistik pro zobrazení celkových statistik hráče, žebříčků atd.
- **Zobrazení v uživatelském rozhraní:**
 - Nová sekce "Statistiky" nebo rozšíření uživatelských profilů a detailů serverů.
 - Grafy, tabulky, žebříčky (leaderboards).

8.2 Podpora dalších herních titulů

- Rozšíření enumerace `GameType`.
- Implementace nových `IGameServerInfoStrategy` pro dotazování specifických protokolů nových her.
- Příprava a testování nových LinuxGSM Docker obrazů pro tyto hry.
- Úprava UI pro zobrazení specifik pro nové hry.

8.3 Zátěžové testování a optimalizace výkonu

- Provést zátěžové testy API a SignalR hubů pomocí zkušební LAN party či externího nástroje např. **k6**.

8.4 Vylepšení UI/UX

- Na základě zpětné vazby od uživatelů vylepšovat použitelnost a vzhled.
- Přidání pokročilejších filtrů a možností řazení v seznamech serverů/anket.
- Vylepšení responzivního designu.

8.5 Pokročilejší správa Docker prostředků

- Možnost detailnější konfigurace parametrů herního serveru při vytváření (např. RCON heslo, název serveru v hostname, specifické game-parametry).
- Správa síťových nastavení Dockeru z UI.
- Zálohování a obnova konfigurací serverů (volumes).

8.6 Možnosti integrace s externími systémy

- **Discord Bot:** Pro notifikace o stavu serverů, anketách nebo pro ovládání některých funkcí přímo z Discordu.

9 Závěr

9.1 Shrnutí

Tato dokumentace poskytuje detailní technický popis aplikace Respawn. Byly popsány funkční a nefunkční požadavky, celková architektura systému, technologický stack a návrh klíčových komponent backendu i frontendu. Důraz byl kladen na použité návrhové vzory, mechanismy autentizace a autorizace, real-time komunikaci pomocí SignalR a interakci s Dockerem pro správu herních serverů. Dále byly zahrnuty diagramy znázorňující různé aspekty systému, stručná uživatelská příručka, doporučení pro provoz, údržbu a náměty pro budoucí vývoj.

Vývoj aplikace odevzdáním semestrální práce nekončí. Cílem je implementovat zbylé navržené požadavky pro zpříjemnění zážitků ze zmíněných LAN party a splnit cíl "bring order to chaos".

10 Přílohy

10.1 Kompletní specifikace API (Swagger/OpenAPI)

Specifikace API je generována dynamicky pomocí Swashbuckle.AspNetCore a je dostupná na endpointu `/swagger` běžící instance RespawnApi (např. <https://localhost:5207/swagger> při lokálním vývoji).

10.2 Seznam konfiguračních parametrů

Klíčové konfigurační parametry jsou spravovány v souborech `appsettings.json` (pro backend) a `.env` (pro frontend).

Backend (`appsettings.json`):

- `ConnectionStrings:DefaultConnection`: Připojovací řetězec k databázi.
- `JwtSettings:Key`: Tajný klíč pro podepisování JWT. **Musí být silný a držen v tajnosti!**
- `JwtSettings:Issuer`: Vydavatel JWT.
- `JwtSettings:Audience`: Cílová skupina JWT.
- `JwtSettings:DurationInHours`: Doba platnosti JWT v hodinách.
- `Logging:LogLevel:Default`: Úroveň logování.
- `AllowedHosts`: Seznam povolených hostitelů.
- `DockerSettings:DockerApiUri` (pokud je explicitně konfigurováno): URL Docker API.

Frontend (`.env` soubory):

- `VITE_API_URL`: Základní URL adresa backendového API (např. <https://localhost:5207/api>).
- Další proměnné specifické pro build proces Vite nebo konfiguraci frontendu.

Pro produkční nasazení by citlivé konfigurační hodnoty (zejména `JwtSettings:Key` a `connection strings`) měly být spravovány pomocí bezpečnějších mechanismů, jako jsou např. environmentální proměnné.